Plant A voices

How the noise, the beep and the flute are made

mami-sound — `src/noise.zig`, `src/tone.zig`, `src/sampler.zig`

1 What the three voices share

Plant A can sound in three ways, chosen with `--voice=drone|flute|beep` (`src/cli.zig`). All three answer the same question — *what pitch is the plant asking for, right now* — and differ only in what makes the sound at that pitch.

Voice	File	Sound source
`drone`	`noise.zig`	white noise through a resonant bandpass; pitch = filter centre
`beep`	`tone.zig`	one sine oscillator; pitch = phase increment
`flute`	`sampler.zig`	twelve recorded flute notes, replayed at a fractional rate

Every voice implements the same call:

```
pub fn render(self: *V, out: []f32, ecg_volts: f32, touched: bool) void
```

and **adds** into `out` rather than overwriting it. That is the whole mixer: the loop in `src/main.zig` zeroes a block, then renders A, B and C into it in sequence.

1.1 The units you are working in

Term	Meaning here
sample	one `f32`, nominally in $[-1, 1]$; anything past that is clamped
sample rate f_s	`root.zig: sample_rate = 44100` — samples per second
frame	one sample time; mono, so frame = sample
block	`root.zig: block_frames = 512` ≈ 11.6 ms. Sensors are polled once per block
`volts_max`	`sensors.zig` — 3.3 V, full scale of the ECG probe

The block is the **control rate** (86 Hz) and the sample is the **audio rate** (44100 Hz). The ECG reading only changes once per block, which is a staircase. Everything in Section 3 exists to turn that staircase into a glide.

2 Frequency: volts to hertz

2.1 The mapping is exponential, not linear

`noise.zig: freqFromVolts` and `tone.zig: freqFromVolts` are the same function:

$$t = \operatorname{clamp}\left(\frac{v}{v_{\max}}, 0, 1\right), \quad f = f_{\min} \cdot \left(\frac{f_{\max}}{f_{\min}}\right)^{t}$$

```
pub fn freqFromVolts(volts: f32) f32 {
    const t = std.math.clamp(volts / sensors.volts_max, 0.0, 1.0);
    return freq_min * std.math.pow(f32, freq_max / freq_min, t);
}
```

Why exponential. Pitch perception is logarithmic: 100⟶200 Hz and 1000⟶2000 Hz are both one octave, and sound like the same distance. A linear map ($f = f_{\min} + t(f_{\max} - f_{\min})$) would spend most of its travel

in the top octave and the bottom volt would be inaudibly cramped. With the exponential map, equal voltage steps are equal **musical** steps.

The full range spans $\log_2\left(\frac{2000}{120}\right) \approx 4.06$ octaves, so with $v_{\max} = 3.3$ V, one volt of ECG $= 1.23$ octaves $\approx$ 14.8 semitones.

2.2 The clamp is not optional

`t` is clamped before the power. A negative reading or a spike past full scale would otherwise produce a frequency below `freq_min` or above `freq_max` — and above $\frac{f_s}{2}$ (22050 Hz) a sine aliases and the filter goes unstable. The clamp is the only thing standing between a wild sensor and a broken sound.

2.3 `freq_min` / `freq_max`

Constant	Value	What it controls
`freq_min`	120 Hz	Pitch at 0 V. Below ~80 Hz small speakers reproduce nothing
`freq_max`	2000 Hz	Pitch at 3.3 V. *The first number to retune* for a different room

Both the drone and the beep use the same pair **deliberately**, so the two can be compared by ear: switch `--voice` and the pitch does not move, only the timbre.

2.4 The flute maps differently

The flute cannot reach 120 Hz — it only has what was recorded. So `sampler.zig: targetHz` maps onto the **set's own** range:

```
pub fn targetHz(set: []const Prepared, volts: f32, volts_max: f32) f32 {
    const lo = set[0].f0;              // 259.24 Hz
    const hi = set[set.len - 1].f0;    // 1859.59 Hz
    const t = std.math.clamp(volts / volts_max, 0.0, 1.0);
    return lo * std.math.pow(f32, hi / lo, t);
}
```

Same formula, different endpoints — 2.84 octaves instead of 4.06. Expect the flute to sound **less** dramatic across a touch than the drone. That is the set, not the code.

The `f0` values are **measured**, by harmonic product spectrum over each file's sustain, not parsed from the filename. This set's names are two octaves below what it actually sounds, and it is tuned 15–45 cents flat of A440. Trusting the names would put every note out of tune.

3 Smoothing: `smoothingAlpha` and the one-pole filter

This is the single most important parameter in the project. It is what makes the piece a **drone that breathes** rather than a sensor readout.

3.1 The coefficient

```
fn smoothingAlpha(tau_s: f32, sample_rate: u32) f32 {
    const sr: f32 = @floatFromInt(sample_rate);
    return 1.0 - @exp(-1.0 / (tau_s * sr));
}
```

$$\alpha = 1 - e^{-\frac{1}{\tau f_s}}$$

applied once per sample as a one-pole (exponential) lowpass:

```
self.fc += (target_fc - self.fc) * self.alpha;
```

$$y[n] = y[n-1] + \alpha(x[n] - y[n-1]) = (1-\alpha)y[n-1] + \alpha x[n]$$

3.2 Reading τ

τ (`smooth_tau_s`, 1.0 s in all three voices) is the **time constant**: the time to cover $1 - \frac{1}{e} \approx 63.2\%$ of a step.

Elapsed	Fraction of step covered	At $\tau = 1$ s
τ	63.2%	1 s
2τ	86.5%	2 s
3τ	95.0%	3 s
5τ	99.3%	5 s

Equivalent -3 dB corner frequency:

$$f_c = \frac{1}{2\pi\tau} \approx 0.159 \text{ Hz}$$

So the pitch control signal is bandlimited to about a sixth of a hertz. **Anything the ECG does faster than that never reaches the ear as pitch.** That is the slow-envelope extraction, stated as one number: individual heartbeat spikes are rejected; the plant's overall state gets through.

At $f_s = 44100$ and $\tau = 1$: $\alpha \approx 2.268 \times 10^{-5}$.

3.3 Why `1 - exp(...)` and not `1/(tau*sr)`

$\alpha = \frac{1}{\tau f_s}$ is the common shortcut and is nearly identical here (2.268×10^{-5} vs 2.268×10^{-5}). The exponential form is exact for any τ and, critically, **cannot exceed 1**. The shortcut goes unstable when $\tau f_s < 1$; the exponential form degrades gracefully to $\alpha \to 1$ (no smoothing). Free correctness — keep it.

3.4 Retuning τ

Change	Effect
$\tau \downarrow$ (e.g. 0.1 s)	Pitch tracks the ECG closely. Heartbeat spikes become audible warbles. Feels reactive, reads as nervous
$\tau \uparrow$ (e.g. 5 s)	Very slow drift. The plant seems to be thinking. Risk: a short touch ends before the pitch arrives

Rule of thumb: τ must be **well under** the shortest touch you expect (plant A is scripted for 11 s in `sensors.zig`), and **well over** the period of whatever you want to reject.

3.5 One caveat: f32 never quite arrives

The test `smoother converges to its target without overshooting` documents it: near convergence, $(x_{\text{target}} - x)\alpha$ is too small to move an `f32` mantissa, so `x` stops about 0.5% short. 0.5% of a frequency is under 9 cents — inaudible — so it is left alone. Do not "fix" it with a snap-to-target; that reintroduces a step.

4 The gate envelope

Identical in all three voices:

```
const gate_ms: f32 = 150.0;
.gate_step = 1.0 / (gate_ms / 1000.0 * sr),
```

$$\text{gate\_step} = \frac{1}{0.15 f_s} \approx 1.512 \times 10^{-4}$$

Per sample, `env` walks linearly toward 1 (touched) or 0 (released), reaching it in exactly 150 ms.

Why it exists. Multiplying a running oscillator by a hard 0/1 switch is a discontinuity, and a discontinuity is broadband energy — a click. 150 ms is long enough to be click-free and short enough to still feel like a response to touch.

Why linear and not exponential. For a fade this short the shape does not matter perceptually, and linear reaches exactly 0 and exactly 1 in finite time, which is testable (`gate reaches silence and full level in the expected time`).

A released voice fades to 0 and stays there. That is also how `select.zig` disables a plant: it masks the touch to `false`, and the voice simply never opens. No special case anywhere in the render loop.

5 `voice_gain` and the clamp

Every voice ends with:

```
sample.* += std.math.clamp(value * self.env * voice_gain, -1.0, 1.0);
```

`voice_gain = 0.4` in all three voices **and** in `player.zig`. Three voices at 0.4 sum to at most 1.2 in the worst case, but real signals are not correlated, so the sum sits comfortably under 1.0 in practice and the clamp is a safety net, not a limiter. Raising `voice_gain` above 0.4 makes the clamp start to bite, and a clamp that bites is distortion.

The drone's test `output is audible but not pinned to the clamp` asserts $0.02 < \mathrm{RMS} < 0.4$ with fewer than 0.1% of samples clipped. Copy that test shape if you add a voice.

6 Voice 1: the drone (`noise.zig`)

White noise pushed through a resonant bandpass. What you hear as pitch is the filter's centre frequency.

6.1 The noise source

```
const input = rand.float(f32) * 2.0 - 1.0;
```

Uniform white noise in $[-1, 1)$, from a seeded `DefaultPrng`. Uniform, not Gaussian: after a narrow bandpass the distribution of the **input** is irrelevant — only its flat spectrum matters — and uniform is one multiply.

The seed is fixed (`main.zig: seed = 0xC0FFEE`) so every run sounds identical and can be compared by ear.

6.2 The Chamberlin state-variable filter

```
const f = 2.0 * @sin(std.math.pi * self.fc / sr);
self.low  += f * self.band;
const high = input - self.low - damping * self.band;
self.band += f * high;
```

Three simultaneous outputs from two state variables: `low` (lowpass), `high` (highpass), `band` (bandpass). This voice uses the bandpass tap only.

The `f` coefficient. $f = 2\sin\left(\pi \frac{f_c}{f_s}\right)$ is the frequency-warped tuning term. The naive $f = 2\pi\frac{f_c}{f_s}$ is the small-angle approximation of it and drifts sharp as f_c rises. At 2000 Hz the difference is already 0.3% (about 6 cents); the `sin` form keeps the tuning honest across the whole 4-octave range.

Stability. The strict condition is $f < 2$ — damping, but the usual practical ceiling quoted for this topology is $f_c < f_s/6 \approx 7350$ Hz, above which the tuning and the resonance stop behaving. `freq_max` of 2000 Hz gives $f \approx 0.284$ — enormous margin either way. The test `filter stays bounded at both pitch extremes` guards this at both ends of the range.

The `f` is recomputed every sample because `fc` moves every sample. A `sin` per sample is not cheap, but at 44.1 kHz mono it is nothing.

6.3 `damping` (= $\frac{1}{Q}$)

```
const damping: f32 = 0.08;   // Q = 12.5
```

The only feedback term. It sets bandwidth:

$$Q = \frac{1}{\text{damping}} = 12.5, \quad \text{bandwidth} = \frac{f_c}{Q}$$

At $f_c = 1000$ Hz, the band is 80 Hz wide.

damping	Q	Character
0.5	2	Wide, obviously noisy. Pitch is a colour, not a note
0.08	12.5	Current. A whistle with breath around it — the installation's sound
0.02	50	Nearly a pure tone. Rings; sounds synthetic; risks self-oscillation artefacts

This is the **timbre** knob. `freq_max` is the pitch knob.

6.4 `makeup`

```
const makeup: f32 = 9.0;
const value = self.band * damping * makeup * self.env * voice_gain;
```

A narrow bandpass passes only a sliver of the noise spectrum, so its output is tiny. Two corrections stack:

1. `* damping` — the bandpass tap of an SVF has gain Q at resonance; multiplying by $\frac{1}{Q}$ normalizes it back to unity.
2. `* makeup = 9.0` — an empirical trim so the drone sits at roughly the same perceived loudness as the beep and the flute.

`makeup` is **verified by test**, not by ear alone: change `damping` and the RMS test will tell you whether `makeup` still holds.

6.5 `crossingRate`: how the drone's pitch is checked

A zero-crossing count is a cheap stand-in for a pitch detector. For narrowband noise it lands near $2f_c$ (two crossings per cycle). The helper lets `freq_min` / `freq_max` be verified from the **rendered audio**, not just from the mapping function.

7 Voice 2: the beep (`tone.zig`)

The simplest voice, and the one that shows the pitch mapping most plainly — nothing is filtered, sampled or looped.

7.1 Phase accumulation

```
const value: f32 = @floatCast(@sin(self.phase));
self.phase += 2.0 * std.math.pi * @as(f64, self.hz) / sr;
if (self.phase >= 2.0 * std.math.pi) self.phase -= 2.0 * std.math.pi;
```

$$\varphi[n] = \varphi[n-1] + \frac{2\pi f[n]}{f_s}$$

Why accumulate instead of computing $\sin(2\pi ft)$. This is the central lesson of the file. With a **moving** frequency, $\sin(2\pi ft)$ recomputed after a frequency change jumps the phase — at $t = 10$ s, a change from 440 Hz to 441 Hz moves the argument by $2\pi \cdot 10$ radians. That is a discontinuity, and a discontinuity clicks. Accumulated phase is continuous by construction: the frequency change alters the **slope**, never the value.

Same reason the flute never restarts a sample mid-note, and the drone never resets its filter state.

7.2 `f64` phase

`phase` is `f64` while everything around it is `f32`. Phase is **accumulated without bound** before wrapping — error compounds every sample, 44100 times a second. `f32` would drift audibly over minutes. The `sin` result is cast back to `f32`; only the accumulator needs the precision.

7.3 Wrapping

Subtract 2π rather than `@mod`: one conditional subtract is exact, cheap, and sufficient because the increment is always smaller than 2π (it would take $f > f_s$ to break that).

7.4 `amplitude`

```
const amplitude: f32 = 0.35;
```

A sine's RMS is $\frac{A}{\sqrt{2}} \approx 0.247$. Chosen to land near the drone's and the flute's measured RMS, so `--voice` switches timbre without switching volume.

8 Voice 3: the flute (`sampler.zig`)

The ECG picks a target frequency exactly as before. Instead of **synthesizing** that pitch, this finds the recorded note nearest it and replays that recording at a fractional rate so it lands on the target exactly.

Nothing in the file knows it is a flute. Point it at another array of `{ path, f0 }` and it plays that instrument.

8.1 Playback rate

```
const rate: f64 = @as(f64, self.hz) / @as(f64, self.set[self.head.idx].f0);
self.head.advance(self.set, rate);
```

$$\text{rate} = \frac{f_{\text{target}}}{f_0}$$

Rate 1.0 = original pitch and speed. Rate 1.12 = one whole tone up, and 12% faster. This is tape-speed pitching: pitch and duration are locked together, and the formant structure moves with the pitch. Stretch far enough and a flute turns into a kazoo — hence the next section.

8.2 Why twelve samples, spaced 2–4 semitones

Neighbouring recordings sit 192–397 cents apart, so the nearest one is never more than about **two semitones** away, and the rate stays within $[0.89, 1.12]$. Small enough that the instrument still sounds like itself.

Boundaries fall on the geometric mean of neighbouring `f0` because `nearestIndex` compares in **cents**, not hertz:

```
fn cents(hz: f32, ref: f32) f32 {
    return 1200.0 * std.math.log2(hz / ref);
}
```

$$\text{cents} = 1200 \log_2\left(\frac{f}{f_{\text{ref}}}\right)$$

1200 cents = one octave, 100 cents = one semitone. Comparing in hertz would make "nearest" mean nearest **arithmetically**, which biases every choice toward the higher sample.

8.3 `hysteresis_cents` = 30

```
if (here - there > hysteresis_cents) self.crossTo(cand);
```

A target sitting exactly on a zone boundary would otherwise flicker between two recordings as the smoothed pitch jitters across it — a machine-gun of crossfades. The neighbour must be **30 cents closer**, not merely closer, before the voice moves. 30 cents is under a third of a semitone: the extra stretch it permits is inaudible, the flicker it prevents is not.

8.4 `zone_fade_ms` = 50 and phase-aligned crossfades

When the zone does change, `crossTo` starts a 50 ms crossfade (zone_step $= \frac{1}{0.05 f_s} \approx 4.54 \times 10^{-4}$ per sample). Three things happen, in order, and each is load-bearing:

1. **The incoming head enters at the same loop phase as the outgoing one**, never at the attack. A pitch change mid-note must not re-articulate the note.
2. **The entry point is nudged by up to one period to maximise correlation** (`align_window = 512` samples compared). Both heads sound the **same** pitch during the fade, so they are coherent and their relative phase is fixed for the fade's whole length. Left to chance, that phase can be opposition — and two coherent signals in opposition cancel into a **hole** in the middle of the crossfade. 512 samples spans three periods of the lowest note in the set.

3. **The fade is linear, not equal-power.** Equal-power ($\sqrt{t}$ curves) is correct for **uncorrelated** material, where powers add. Step 2 has just made these two correlated and in phase, so their **amplitudes** add, and linear is the shape that holds the level flat. Using equal-power here would produce an audible **bump** in the middle instead of a hole.

That triple — same phase, aligned, linear — is the general recipe for crossfading two copies of the same pitch.

8.5 `attack_seconds` = 0.4 and the loop

```
buf = [ attack (0.4 s) | loop region (crossfaded at its head) ]
        ^ pos 0                ^ loop_start
```

A fresh touch starts at position 0 so the **breath of the attack** is heard — the part of a flute note that makes it recognisable. After that, playback lives in the loop region forever.

`wrapPos` subtracts whole loop lengths, so one `f64` position covers the whole life of the note with no branch on "am I looping yet".

8.6 `bestLoopLen`: choosing the loop length

```
const score = dot / @sqrt(tail_energy);
```

A loop whose length is **not a whole number of periods** folds its tail onto its head out of phase, and the two cancel — a hole, once per loop, forever. So the code searches back over one period for the length whose tail best matches the head.

Scored by **normalized** correlation ($\text{dot} \;/\sqrt{\text{tail energy}}$) rather than raw dot product, so a merely loud stretch of audio cannot win on level alone.

8.7 `loop_fade_seconds` = 0.04

The material **after** the loop end is folded over the loop's start:

```
loop[i] = loop[i] * t + raw[attack + loop_len + i] * (1.0 - t);
```

40 ms is long enough to hide the seam, short enough not to smear the tone. Linear again, and for the same reason as before: `bestLoopLen` has already put the two ends in phase, so they add.

8.8 `target_rms` = 0.25

```
const gain: f32 = @floatCast(target_rms / rms);
```

The raw set spans RMS 0.305 to 0.551 — a 5 dB spread that would **step in loudness at every zone crossing**. Every recording is brought to a shared RMS at load time, so a zone change is heard as a change of pitch only.

RMS, not peak: peak normalization is hostage to a single transient sample, RMS tracks what the ear calls loudness.

8.9 Linear interpolation in `readAt`

```
return a + (b - a) * frac;
```

The read position is fractional, so a sample must be invented between two stored ones. Linear is cheap and, at these small rate ratios (0.89–1.12), its error sits far enough down to be masked by the flute's own noise floor. Pitching **up** by a large factor with linear interpolation would alias audibly — another reason the sample set is dense.

8.10 Retrigger on a new touch

```
if (touched and !self.prev_touch) {
    self.hz = target;                                   // no glide in
    self.head = .{ .idx = nearestIndex(...), .pos = 0.0 };  // start at attack
    self.fade = 1.0;
}
```

Rising edge only. A new touch is a **new note**: the pitch jumps to the current reading rather than gliding up from wherever the last note ended, and playback starts at the attack. During a **held** touch the smoother takes over and the pitch glides as everywhere else.

Note the asymmetry with the drone and beep, which have no notion of a new note at all — they only ever glide.

9 Every tunable, in one table

Constant	File	Value	Turn it to change
Shared / pitch			
`sample_rate`	`root.zig`	44100	Everything. Don't
`block_frames`	`root.zig`	512	Sensor poll rate (86 Hz) and latency (11.6 ms)
`freq_min`	`noise`, `tone`	120 Hz	Pitch floor at 0 V
`freq_max`	`noise`, `tone`	2000 Hz	Pitch ceiling at 3.3 V. *Retune this first*
`smooth_tau_s`	all three	1.0 s	How fast pitch follows the plant. The core gesture
`gate_ms`	all three	150 ms	Touch/release fade length
`voice_gain`	all four	0.4	Headroom before the clamp
`volts_max`	`sensors`	3.3 V	ADC full scale. Hardware fact, not taste
Drone only			
`damping`	`noise`	0.08	Timbre: whistle (↓) vs. wind (↑)
`makeup`	`noise`	9.0	Level trim after the bandpass. Re-check the RMS test
Beep only			
`amplitude`	`tone`	0.35	Level, matched to the other two voices
Flute only			
`flute[]`	`sampler`	12 notes	The instrument, and the pitch range
`attack_seconds`	`sampler`	0.4 s	How much breath a new note has
`loop_fade_seconds`	`sampler`	0.04 s	Loop seam smoothing
`target_rms`	`sampler`	0.25	Shared level across recordings
`zone_fade_ms`	`sampler`	50 ms	Crossfade length at a zone change
`hysteresis_cents`	`sampler`	30	Resistance to zone flicker
`align_window`	`sampler`	512	Phase-alignment search accuracy

10 Formula reference

Quantity	Formula
Volts to pitch	$f = f_{\min}(f_{\max}/f_{\min})^{\mathrm{clamp}(v/v_{\max},0,1)}$
Smoothing coefficient	$\alpha = 1 - e^{-1/(\tau f_s)}$
One-pole step	$y[n] = y[n-1] + \alpha(x[n] - y[n-1])$
Smoother corner	$f_c = 1/(2\pi\tau) \approx 0.159$ Hz
Gate / zone step	$1/(T_{\text{fade}} f_s)$
SVF tuning	$f = 2\sin(\pi f_c / f_s)$
SVF resonance	$Q = 1/\text{damping}$, bandwidth $= f_c/Q$
Phase increment	$\Delta\varphi = 2\pi f / f_s$
Playback rate	$\text{rate} = f_{\text{target}}/f_0$
Interval in cents	$1200\log_2(f/f_{\text{ref}})$
Sine RMS	$A/\sqrt{2}$

11 Three rules that generalise

1. **Never step a control value — smooth it.** Every parameter the sensor touches goes through `smoothingAlpha` before it reaches the audio. Steps click.
2. **Never step the signal either.** Accumulate phase, don't recompute it. Fade gates, don't switch them. Crossfade samples, don't cut them. Enter a new recording at the phase you left the old one.
3. **Choose the fade shape from the correlation.** Correlated and in phase → linear (amplitudes add). Uncorrelated → equal-power (powers add). Getting this backwards gives a hole or a bump in the middle of every fade — the flute's loop fold and its zone crossfade are both linear precisely because both are phase-aligned first.